Query-Aware DVFS For GPU-Accelerated Relational Databases

Olivier Michaud
McGill University
Montréal, Québec, Canada
olivier.michaud2@mail.mcgill.ca

Ricardo Macedo
INESC TEC & U. Minho
Braga, Portugal
ricardo.g.macedo@inesctec.pt

Oana Balmau
McGill University
Montréal, Québec, Canada
oana.balmau@mcgill.ca

Bettina Kemme
McGill University
Montréal, Québec, Canada
bettina.kemme@mcgill.ca

Abstract

GPU-accelerated database technology has advanced rapidly, exploiting the massively parallel capabilities of GPUs to speed up query execution. However, GPUs are power-hungry, leading to high energy consumption. With the popularity of GPUs within data centers to handle ML workloads, dynamic voltage-frequency scaling (DVFS) has become a popular mechanism to slow down GPUs when they are not needed, enabling significant energy savings. In this paper, we explore the potential for DVFS for GPU-accelerated database systems. We propose a fine-grained scaling mechanism that takes into account the compute needs of individual operators within SQL queries and then scales the workload dynamically to provide the GPU power when needed, but reduces frequency otherwise. We materialize this idea in OpPilot, a DVFS planner integrated into Sirius, a GPU-accelerated database. Our experiments show that OpPilot achieves energy savings of up to 40% with minimal performance degradation.

Keywords

GPU-accelerated, frequency scaling, DVFS, energy, database

1 Introduction

In recent years, database systems (DBS) designed for GPUs have undergone significant advances, demonstrating substantial gains in both execution speed and the ability to handle increasingly complex analytical workloads. Modern GPU-accelerated databases leverage massive parallelism and specialized execution models to dramatically reduce query latency compared to traditional CPU-based systems. Despite these improvements, an important and largely neglected dimension of GPU-based database research is energy efficiency. GPUs are inherently power-hungry devices and can consume considerable amounts of energy during execution, raising concerns about environmental impact.

Energy-proportional computing has long been recognized as a design goal for large-scale systems [1]. A wide range of energy-aware mechanisms have been developed since then. One prominent approach is Dynamic Voltage and Frequency Scaling (DVFS), which adjusts the operating frequency of a given processing unit. By throttling when full performance is not required, DVFS can reduce energy consumption without necessarily affecting classic performance metrics such as latency. There exists ample research on how to apply CPU-based DVFS, especially for cloud-based workloads [33], and for database systems [10, 21, 29, 39, 42]. More recently, the focus has shifted to DVFS for GPUs as they are now the major energy consumer in data centers, in particular for ML workloads. While some of these approaches work at the coarse level of a model or an application [24, 37], fine-grained approaches often consider specific ML operators, take into account the important batch size parameter or the intrinsics of ML parallelization [4, 35, 45]. An exception to this ML work is the general-purpose dynamic scaling approach proposed by GEEP-AFS [48]. However, as a generic profiling tool, it cannot fully capture the characteristics of the workload.

This ML-driven GPU scaling work, while not directly applicable to database systems, has motivated us to have a closer look at DVFS for GPU-based database systems. Similar to ML execution, we observe that not all of the energy consumed by the GPU translates into performance. In fact, a fair number of database operators (*e.g.*, scans, projections) do not perform complex computation but are instead limited by memory accesses and thus do not fully utilize the GPU's computational capacity. This is quite different from the most common ML operators, whose floating-point operations are GPU-hungry. Furthermore, database queries often process large volumes of data, which might not fit into the GPU memory. This leads to frequent data transfer between CPU and GPU, during which the GPU is not fully utilized. Finally, most GPU database systems also partially execute on the CPU, leading to phases where the GPU is idle.

Together, these limitations motivate us to analyze and quantify the potential of dynamic GPU throttling for database systems. For our analysis, we use Sirius [44], a GPU-accelerated database system. At its extreme, we could perform operator-aware frequency scaling so that each operator within a query executes at the lowest GPU frequency level that does not impact its runtime. However, not all operators execute long enough to amortize the latency cost of a frequency transition, which can be significant.

To address this trade-off, we propose an *operator-aware DVFS strategy* that *(i)* determines offline, for each operator, the lowest clock frequency that preserves runtime, grouping operators with similar frequencies into categories; and *(ii)* at runtime, analyzes the query execution plan and groups consecutive operators so that each group's estimated execution time amortizes the latency of a frequency transition. The group sizes also take into account that runtime estimation can be noisy. Each group is then run at a specific frequency depending on the categories of the operators it contains. We integrate this strategy into OpPilot, a DVFS planner that sits between Sirius's physical query planner and the execution engine: it consumes the physical plan produced by Sirius, annotates it with per-group frequency-setting instructions based on the categorization and grouping above, and outputs an extended physical plan that Sirius executes directly, issuing frequency changes to the GPU as each group begins.

We evaluate OpPilot on two GPU architectures (*i.e.*, NVIDIA V100 and RTX 5000 Ada) against several baselines on the TPC-H

EDBT '27, Lille (France)
© 2026 Copyright held by the owner/author(s). Published on OpenProceedings.org under ISBN 978-3-89318-106-3, series ISSN 2367-2005. Distribution of this paper is permitted under the terms of the Creative Commons license CC-by-nc-nd 4.0.

benchmark [38]. OpPilot achieves energy savings of up to 40% while keeping response-time impact under or close to 5%. We identify which operators benefit most from throttling, and provide a critical analysis of when the benefits of dynamic throttling fail to outweigh its complexity relative to simpler methods.

2 Related work

GPU-accelerated database systems. There exists a large body of work on GPU-accelerated query execution, exploiting the highly parallel compute model of GPUs to provide efficient operators such as joins, aggregation, and sorting [2, 6, 14, 15, 23, 32, 41, 44, 46]. Other work focuses on heterogeneous CPU-GPU implementations to improve resource utilization or overcome the limited CPU-GPU bandwidth [3, 13, 17, 22, 25, 26, 47]. Finally, distributed implementations use multiple GPUs to improve scalability [7, 43, 44]. In our work, we use the GPU-accelerated query engine Sirius [44]. Its TPC-H evaluation shows execution time is dominated by different operators depending on the query (*e.g.,* `joins` in join-heavy queries, or `filters` in highly selective queries). This operator-dependent design motivates OpPilot's operator-level categorization. Sirius, however, targets only runtime and cost efficiency. OpPilot extends Sirius's architecture with a DVFS planner while reusing its physical query plans and operator pipeline. We describe Sirius in detail in §4.

Dynamic Voltage-Frequency Scaling (DVFS). DVFS is a well-established energy-saving technique that reduces the voltage and/or frequency of a given processing unit (CPU or GPU). This reduces power draw, directly impacting energy consumption.

DVFS for GPUs. As newer GPU hardware reaches power consumption levels of up to 1400W [27], controlling power draw through DVFS becomes crucial when the hardware is not fully utilized. Applying DVFS effectively, however, is not trivial, as frequency, energy consumption, and runtime penalty are not linearly related, and energy benefits are heavily application-dependent [8].

A considerable body of work quantifies how frequency scaling can reduce energy consumption. Initial work focused on the optimal frequency for an entire application [16, 24, 37]. From there, several proposals target fine-grained DVFS for ML workloads, having a deep look into what impacts ML. Zeus [45] balances batch size (an important parameter in ML workloads) with the GPU's power limit. Perseus [4] applies DVFS to distributed ML to reduce the frequency of GPUs that are underutilized because they are assigned less computational work; this can frequently happen when ML uses data and pipeline parallelism. Conceptually related to OpPilot, [35] analyzes the DVFS potential for individual kernels within an ML pipeline. However, they still treat batch size and parallelism as input parameters for the analysis where we propose categorization to ensure feasibility and operator grouping to handle the frequency scaling overhead.

Methodology-wise, these approaches take advantage of the iterative nature of ML to dynamically adapt to a specific ML model. During the first iterations of the learning process, they determine the optimal frequencies and/or determine how and where the GPU is underutilized, and then react to it. Such an online approach is needed as the execution patterns of the various ML models can differ significantly. In contrast, DBMS have a relatively small fixed set of operators that can be analyzed in an offline phase. Furthermore, the full execution plan of a SQL query with all operators and estimated runtimes is known at the beginning of the execution, which allows us to determine in advance the exact frequencies to use for each query.

GEEPAFS [48] is an application-agnostic tool that leverages DVFS while offering performance assurance. Using online profiling, *GEEPAFS* tunes the GPU's frequency based on metrics obtained from the GPU driver rather than the application itself.

DVFS for databases. Energy efficiency has received attention in the database community in prior work. A survey on early work [9] includes several CPU-based DVFS proposals. This includes evaluating the impact of DVFS [21], and tuning frequencies online while controlling throughput [39, 42]. Work on calibrating on a per-query [10] or per-operator [29] basis considers memory access patterns, parallelism, and thread scheduling. [19] target memory-intensive workloads and [12] extend to energy-aware control at the storage layer. Reinforcement learning has been shown to be suitable for frequency tuning when configurations change [31]. Taken together, these works show that hardware tuning should be handled by the database itself to fully exploit its potential for energy efficiency [18], since application-agnostic approaches lack workload knowledge to do so effectively.

To the best of our knowledge, all existing work targets database systems executing on CPU. In contrast, our work explores GPU-accelerated database systems for which the energy characteristics and the performance-energy relationship are still unexplored.

3 Query-aware DVFS

Our idea is to perform fine-grained dynamic frequency scaling within a query, reducing overall energy consumption while ensuring that query response time does not increase beyond a given threshold. Our per-operator analysis of Sirius [44] shows that operators within a query exhibit different compute demands, and therefore offer different potential for frequency scaling. Throttling the GPU during the execution of a compute-intensive operator incurs a significant latency penalty, while other phases of query execution can tolerate lower frequencies. This observation motivates frequency scaling on a query operator basis. As there is only a restricted set of operators, it is feasible to determine the optimal frequency for each of them in an offline phase. However, frequency transitions are not instantaneous. A comparison of various GPU architectures [40] has shown that transition latency ranges from hundreds of microseconds to hundreds of milliseconds. Given that individual database operators can themselves be short-lived, per-operator scaling is thus not always feasible in practice. We address this with two mechanisms.

Operator categories. We first assign each operator to a scaling category. All operators in a given category are scaled to the same frequency. The frequency chosen for each category depends on the underlying hardware and the operators it contains, so that one only needs to switch between a limited number of frequencies. Previous work [48] has shown that the relationship between the runtime and the GPU's clock frequency depends on whether a workload's memory latency can be hidden behind other instructions. We build on this insight to define our categorization scheme. Given an operator whose data already resides in GPU memory, we assign it to one of the following three categories.

- **Compute-bound:** operators that perform complex computations (*e.g.,* multiplications, complex comparisons) are dominated by computation latency rather than memory access. Reducing the clock frequency directly increases runtime.
- **Memory-bound:** operators whose computation is simple relative to their memory access pattern are dominated by memory latency. Reducing frequency has little impact on runtime, as the increased computation time is hidden by memory latency.

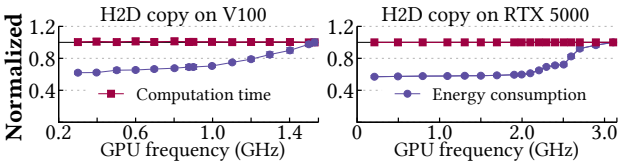


Figure 1: Host-to-device data transfer.

- **Balanced:** operators that fall between compute- and memory-bound can tolerate a moderate frequency reduction without impacting runtime, though the achievable reduction is smaller than for memory-bound operators.

Grouping. Categories alone are insufficient because an execution may alternate between operators of different categories. For instance, if a short *memory-bound* operator executes between two *compute-bound* operators, there may not be enough time to throttle the GPU for its execution before scaling back up for the next operator. As such, at runtime, given a sequence of operators executing a query, we group neighboring operators so that the estimated execution time of the group exceeds the time required to switch frequencies. A grouping policy also takes into account that runtime estimation can be noisy. All operators within the group then execute at the same frequency for which we have experimented with conservative and opportunistic choices (§4.3).

Memory copies and CPU execution. We identify two additional situations in which frequency scaling is not only possible but can be applied quite aggressively. First, since the database may be larger than GPU memory, query input data must often be copied from CPU to GPU memory, and query results must likewise be transferred back to CPU over the PCIe link that connects the two. This data transfer can take a significant amount of time, yet requires little GPU power. Figure 1 shows, for two different GPUs, the computation time and energy consumption of a CPU-to-GPU copy across increasing GPU frequencies (normalized to values at maximum frequency): transfer time remains constant across all frequencies, while energy consumption at the lowest frequency level is 40% lower than at the maximum level. Second, GPU-accelerated databases often offload only part of their execution to the GPU; tasks such as query planning and data pre-processing remain on the CPU.

All these situations enable aggressive frequency down-scaling, motivating an additional scaling category, termed **CPU-bound**, which is not tied to specific operators but to phases in which only data transfer or CPU-side computation occurs.

4 OpPilot: DVFS implementation in Sirius

We design and integrate our query-aware DVFS strategy, which we call OpPilot, into open-source Sirius, a GPU-native SQL engine [34, 44]. Sirius can be integrated with existing database systems, among them DuckDB [5], to replace the query engine. Query execution is illustrated in Figure 2. DuckDB generates a query execution plan based on the Substrait [36] format. This plan is ingested by Sirius to create a physical plan, represented as a sequential pipeline of operators[1]. The operator pipeline for Q12 of the TPC-H benchmark is shown in the first column of Table 2 and the start of its execution on CPU and GPU in Figure 2. The table scan operators process input data on the CPU and move it to GPU memory if it is not already on the GPU (OP1 and OP6 in the figure). GPU memory has two regions, one for processing and another for input and output data. Region size is configurable to accommodate queries with different requirements. For each pipeline, a CPU-based coordinator thread maintains the state of query execution (*e.g.*, operator's input and output) and pushes relevant data into the operators. Each operator can make several calls to execute kernels on the GPU.

[1]In [44], a query consists of several pipelines that could be executed concurrently, but the code version we used did not (yet) have that feature.

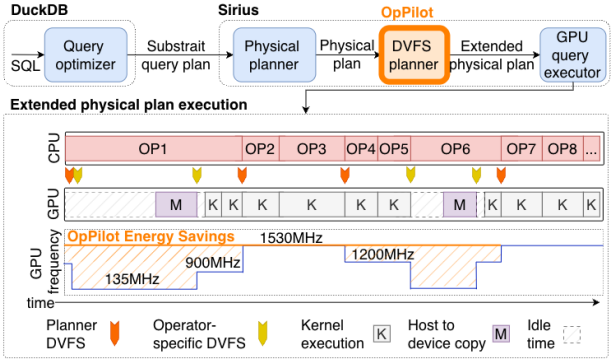


Figure 2: System diagram and query execution as seen from the CPU and the GPU. Time moves from left to right.

4.1 DVFS planner

OpPilot extends the database system architecture with a DVFS planner, as shown in orange in Figure 2. It takes as input Sirius' physical query plan and outputs an extended plan with instructions to set the GPU clock to a specific frequency. The planner works in two main steps. First, taking into consideration the category of each operator in the plan (determined during an offline phase as outlined in §4.2), the planner groups operators to ensure sufficient runtime between frequency changes taking runtime estimation errors into account, and selects the group's frequency (§4.3). It then performs a deeper inspection of the specific operator that prepares and transfers input data to GPU memory, which exhibits significant *CPU-bound* phases (§4.4).[2] Based on this analysis, the planner injects the corresponding frequency-setting commands into the plan. Figure 2 illustrates these injected calls and the resulting GPU clock frequency over time.

4.2 Threshold-based operator classification

We first determined, for each operator, the lowest frequency at which it can still execute within a given performance threshold we have set at 5%. Energy was measured using the NVIDIA NVML tool. We then sorted operators by frequency and clustered them into the three categories: *compute-bound* (5 operators), *balanced* (10 operators), and *memory-bound* (6 operators). For each of these three categories, we select the maximum frequency among its operators, ensuring the performance threshold is met for all of them. For the *CPU-bound* category, we select the lowest frequency supported by the hardware, motivated by the results shown in Figure 1. Table 1 reports the frequency values for the NVIDIA V100 and RTX5000 Ada GPUs.

[2]`TABLE_SCAN+GetDataDuckDB` prepares and transfers input data from CPU to GPU. `RESULT_COLLECTOR+Sink` transfers results from GPU back to CPU memory. We ignore the latter because of the limited result set size of TPC-H benchmark queries.

Table 1: Frequencies assigned to each category.

Category	V100	RTX5000 Ada
COMPUTE_BOUND	1530MHz	3105MHz
BALANCED	1200MHz	2205MHz
MEMORY_BOUND	900MHz	1500MHz
CPU_BOUND	135MHz	210MHz

4.3 Operator grouping

We use a simple heuristic, which we consider sufficient to provide first insights into the impact of query-aware DVFS.

Grouping algorithm. The planner performs a single pass through the operator pipeline and forms groups based on greedy accumulation. We explain the process along Table 2, where the second column shows each operator's category, the third its resulting (ideal) frequency on V100, and the fourth the actual frequency used after grouping. As the plan is traversed, operators are added to the current group as long as they share the category of the operators already in the group. When the next operator's category differs, we approximate the runtime of the current group based on the cardinality estimation of the input data provided by the query planner for each query, combined with the physical size of the columns. If the estimated runtime exceeds the *time interval* required for DVFS, the group is closed and its frequency set to that of its operators, and we continue scanning the plan with a new group (in Table 2, this occurs after operators 1 and 3). Otherwise, operators are added to the group as long as the group as long as they fit within the time interval (operators 4-6 and 7-14 in Table 2 form two such groups).

Determining an appropriate time interval for a group was challenging. On our hardware, transitioning between two frequencies could last up to 3 milliseconds. Given that even the shortest TPC-H queries took many milliseconds, we expected a huge potential for fine-grained groups. However, in practice, runtime variation between operators was substantial, with several running well below a millisecond. Furthermore, our cardinality-based runtime estimation was noisy and with low accuracy. Cardinality estimation [11, 30] and runtime estimation [20] are known to be challenging, so this comes at no surprise. We accommodate for the estimation noise by setting larger time thresholds for grouping, and §5 presents results for three different time thresholds. Building better cardinality and runtime estimation models for Sirius was out of the scope for our work.

Conservative vs. opportunistic grouping. Our grouping algorithm assigns mixed groups the frequency of the most demanding category in the group. We analyze two configurations: *(1)* the *conservative* configuration is our default and uses all three categories (*compute-bound, balanced, memory-bound*) as discussed above. As soon as a compute-bound operator is in the group, we keep the frequency high to ensure we stay within our performance threshold. However, this can lead to extended periods at high frequency. *(2)* As an alternative, the *opportunistic* configuration only retains the *balanced and memory-bound* categories. In opportunistic, all *compute-bound* operators are re-categorized as *balanced*, so that the GPU is always throttled to some degree.

4.4 Transfer optimizations

The table scan operator, combined with getting data from DuckDB, is categorized as *memory-bound* due to some GPU kernels it executes. However, the operator first pre-processes data on the CPU and then transfers it from CPU to GPU memory. Thus, part of this operator actually belongs to our *CPU-bound* category, either because the GPU performs no work at all or because of the transfer itself across the PCIe bus. Note that an idle GPU might throttle down automatically; however, this does not occur in Sirius, as the GPU maintains a CUDA context throughout.

We estimate the *CPU-bound* time based on the size of the data to be transferred, number of PCIe lanes, per-lane speed, and maximum achievable utilization rate, measured using the NVIDIA bandwidth measurement tool [28]. If the time is considered long enough for throttling, we set the GPU to the CPU_BOUND frequency. In Table 2, this is applied for operators 1 and 6.

Table 2: Query and DVFS plans for TPC-H Q12 on V100.

	Operator	Category	Ideal	Conservative	Transfer
1	TABLE_SCAN	MEMORY_BOUND	900MHz	900MHz	
	+GetDataDuckDB	CPU_BOUND			135MHz
2	TABLE_SCAN+GetData	COMPUTE_BOUND	1530MHz	1530MHz	
3	FILTER	COMPUTE_BOUND	1530MHz	1530MHz	
4	PROJECTION	BALANCED	1200MHz	1200MHz	
5	HASH_JOIN+Sink	BALANCED	1200MHz	1200MHz	
6	TABLE_SCAN+	MEMORY_BOUND	900MHz	1200MHz	
	+GetDataDuckDB	CPU_BOUND			135MHz
7	TABLE_SCAN+GetData	COMPUTE_BOUND	1530MHz	1530MHz	
8	HASH_JOIN	COMPUTE_BOUND	1530MHz	1530MHz	
9	PROJECTION	BALANCED	1200MHz	1530MHz	
10	HASH_GROUP_BY+Sink	BALANCED	1200MHz	1530MHz	
11	HASH_GROUP_BY	BALANCED	1200MHz	1530MHz	
	+GetData				
12	ORDER_BY+Sink	COMPUTE_BOUND	1530MHz	1530MHz	
13	ORDER_BY+GetData	MEMORY_BOUND	900MHz	1530MHz	
14	RESULT_COLLECTOR	MEMORY_BOUND	900MHz	1530MHz	
	+Sink				

5 Evaluation

We compare OpPilot against the following baselines:

- *No throttling*, running the GPU at its maximum frequency.
- *Mem-bound*, where the GPU frequency is capped at that of the memory-bound operators for the entire experiment.
- *Balanced*, analogous to *Mem-bound* but with the GPU frequency capped at the balanced category.
- *GEEPAFS* [48], an online workload-agnostic DVFS solution, which was configured to retain 95% of performance (similar to our 5% performance threshold). GEEPAFS adjusts the frequency every 15 seconds. At the beginning of this interval, the workload is run with varying frequencies (every 200ms a new frequency is tested) to determine the optimal frequency.
- *DVFS-Naïve*, a baseline we implement that uses DVFS to throttle each operator at its ideal frequency (as in Table 1).

For OpPilot, we analyze the following grouping strategies:

- *OpPilot Conservative (Cons.)*, which implements the conservative grouping strategy described in §4.3, evaluated under three grouping time intervals: 10ms, 20ms, and 30ms. These configurations do not apply extreme throttling during CPU-bound phases (*i.e.,* during data transfer).
- *OpPilot Conservative plus transfer (Cons.+T)*, which augments the *Conservative* grouping with additional throttling during CPU-bound phases. We report only for the 30ms time interval.
- *OpPilot Opportunistic plus transfer (Opp.+T)*, which combines the *Opportunistic* grouping strategy described in §4.3 with the CPU-bound throttling optimization, again using 30ms.

Workloads. We use the TPC-H workload with a scaling factor of 30, the largest that does not exhaust GPU memory for any individual query. In a first set of experiments, we ran all the TPC-H queries one after the other. It involves regularly moving data

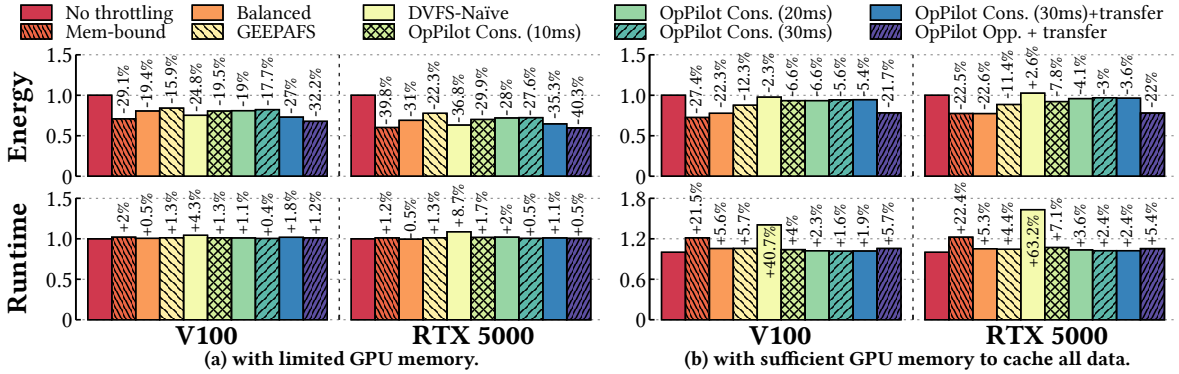


Figure 3: Energy consumption and runtime of the TPC-H workload when GPU memory cannot hold the entire data *(a)* and with all data cached *(b)* on the GPU, normalized to the standard execution without DVFS.

in and out of the GPU based on query requirements, as not all the tables fit in GPU memory. However, for some queries, no data movement is required since the data is already on the GPU from a previous query. This represents a diverse workload, with queries that might run very quickly along with long-running queries that require significant data preparation and movement. A second set of experiments emulates unlimited GPU memory, where all data can be cached in GPU memory. For that, we first bring the data needed for a particular query into GPU main memory and only then start time and energy measurements. Purely in-memory processing puts pressure on the DVFS planner as query execution time is overall much shorter without data transfer.

Hardware configuration. We use two servers. A V100 server with 2× Intel Xeon with 512GB of memory, a NVIDIA V100-SXM2-32 GPU, and a 16x Gen 3 PCIe Link. It runs Ubuntu 20.04.6 LTS with driver version 560.35.05. A RTX5000 server with 2× AMD EPYC 48-core with 512GB of memory, an NVIDIA RTX 5000 Ada Generation GPU, and a 16x Gen 4 PCIe Link. It runs Ubuntu 24.04.4 LTS with driver version 580.159.03. All CUDA code was compiled with toolkit 12.9.

5.1 Limited GPU memory

Figure 3(a) reports aggregated runtime and energy consumption across all 22 TPC-H queries when data is brought into GPU memory whenever needed, normalized to not throttling the GPU at all (which ran 53s and needed 4.13kJ for V100, and 30s/1.83kJ for RTX5000). With the exception of DVFS-Naïve, all scaling strategies incur at most 2% runtime overhead, well within our target threshold. *Balanced*, GEEPAFS, and all OpPilot Conservative and Opportunistic variants have at most three individual queries above the 5% threshold, while *memory bound* has 5/3 queries for V100/RTX5000. In contrast, DVFS-Naïve has an overhead up to 8.7% (for RTX5000) with 13/16 queries (V100/RTX5000) above the 5% threshold. This stems from the overhead of frequent rescaling.

Energy gains, however, vary substantially across strategies. On V100, GEEPAFS provides the smallest energy gain (15.9%), followed by *Cons.*, ranging from 17.7% and 19.5% depending on the grouping interval. Adding aggressive throttling during the *CPU-bound* transfer phase (*Cons.+T*) raises this to 27%, and *Opp.+T* adds another 5%, for an overall energy gain of 32.2% (outperforming DVFS-Naïve with 24.8%). With 30ms time intervals, there are on average 3.6 groups per query: 45% of them have a compute-bound operator and make up around 13% of the overall runtime for *Cons.* By letting these groups run on the balanced frequency, *Opportunistic* can achieve the higher energy savings.

RTX5000 shows a similar pattern, with *Opp.+T* again performing best, exceeding 40% in energy savings. Interestingly, statically capping at the balanced frequency yields slightly higher energy gains than GEEPAFS and *Cons.* (which do not throttle during GPU-bound phases). Similarly, capping at the *memory-bound* frequency slightly outperforms *Cons.+T*. Smaller grouping intervals lead to marginally better energy savings at the cost of higher runtime. *Opp.+T* still surpasses both static configurations. Its aggressive downscaling during CPU-bound periods saves more energy than *Balanced*. Furthermore, as *Opp.+T* runs the SQL operators with higher frequency than *Memory-bound* the resulting shorter runtimes lead to overall lower energy consumption. *Opp.+T* is also again better than DVFS-Naïve.

Overall, *Opp.+T* offers the best trade-off: significant energy gains with minimal performance penalty.

5.2 Unlimited GPU memory

Figure 3(b) shows the runtime and energy costs when having all data cached in GPU memory, again normalized to no throttling (which ran 2.9s on the V100 and 1.9s on the RTX5000, *i.e.*, more than an order of magnitude faster than with memory transfer costs; energy consumption was 477J and 142J, respectively). The trends differ from when data is moved to the GPU.

DVFS-Naïve incurs the largest runtime penalty, exceeding 63% on RTX 5000, followed by *memory-bound* capping at over 20%. *Balanced* capping and *Opp.+T* fall slightly above the performance threshold (but still below 6%). The threshold violations under these aggressive throttling strategies stem from the overall very fast runtimes in this experiment where even a small absolute increase in query time can translate into a large percental penalty. GEEPAFS and *Cons.(10ms)* are each above the threshold for one GPU type, and below for the other, while the remaining *Conservative* alternatives remain comfortably within the threshold. Comparing the 10-30 ms grouping times, runtimes increase as grouping times shrink, since runtime estimation errors have a bigger impact with smaller groups and more frequent frequency adjustments lead to extra overhead.

In terms of absolute numbers, DVFS-Naïve and *Memory-bound* were above the 5% runtime threshold for all 22 queries on both GPU types, *Balanced* and *Opp.+T* for 16/13, *Cons.(10ms)* for 6/10, *Cons.(20ms)* for 4/6 and *Cons.(30ms)* for 2/3 queries (V100/RTX5000).

With DVFS-Naïve and *Memory-bound*, 20 queries were even more than 10% slower, while there were only 6/3 (V100/RTX5000) queries for GEEPAFS, 6/2 for *Balanced* and *Opp.+T*, 4/6 for *Cons. (10ms)*, and at most 3 for the 20ms and 30ms grouping times.

In terms of energy, aggressive *memory-bound* static throttling yields the largest gains. Balanced throttling and *Opp.+T* achieve comparable reductions of 22-23% while GEEPAFS has 11% and the conservative options only up to 7%. DVFS-Naïve has the worst results, saving only 2.3% on V100 and even having an energy penalty of 2.3% on RTX500 due to its very long runtime.

Overall, in compute-heavy environments with limited memory transfer operations, there is a more pronounced trade-off between energy savings and runtime increase. However, given the overall short runtimes of only a few seconds for the entire TPC-H benchmark, we believe that the moderate runtime penalties caused by *Opp.+T* and *Balanced* for some of the queries might be acceptable given the significant energy savings compared to GEEPAFS and the conservative alternatives.

5.3 Discussion

Opp.+T provides the best overall energy savings while also maintaining good performance, though it slightly exceeds the targeted 5% performance threshold when all data is cached on the GPU. The *Cons.* variants, in turn, offer the best performance preservation. DVFS-Naïve is ineffective, as its per-operator granularity is too fine-grained given the delays in frequency transitions.

Transfer optimizations. §5.1 shows the importance of transfer-specific optimizations. When data is not cached on the GPU, the table scan dominates runtime, with a major portion of it leaving the GPU either idle or performing memory transfer. Aggressively throttling during these phases brings substantial energy savings.

Conservative vs. Opportunistic. Although *Conservative*'s three throttling categories appear to provide fine-grained scaling, its grouping requirements prevent it from fully exploiting this potential. Many operators are too short-lived to be scaled individually, and since the most compute-intensive operator in a group determines the group's frequency, energy-saving potential is left unused. This is particularly pronounced when data is cached. The scan operator, though categorized as *memory-bound*, becomes so short in the absence of data transfer that it is often grouped with *compute-bound* operators, leaving little opportunity for throttling.

Opportunistic, by eliminating the highest frequency altogether, keeps the GPU at most at the *balanced* frequency while still allowing reductions during *memory-bound* and *CPU-bound* phases. As a result, it achieves substantially better energy gains, particularly when data is cached. While throttling *compute-bound* operators might lead to 5-10% larger execution times for a fair amount of queries, given the overall fast execution, surpassing the 5% threshold might be acceptable given the significant energy gains compared to the conservative approach.

Overall, we consider *Opportunistic* to achieve the best trade-off between energy consumption and runtime. There are, however, scenarios where *Conservative* is a better choice, including queries with tight SLOs or long-running compute-intensive operations (*e.g.,* large joins). In such cases, the two strategies can be combined, for instance, running in a default *Opportunistic* grouping, but switching to *Conservative* within a group whenever its compute-intensive operators exceed a given time limit.

Time intervals for groups. While we adopt 30ms as the default grouping interval, shorter intervals provide marginally better energy savings at the cost of longer runtimes. Fine-tuning the interval to avoid response time spikes will be crucial.

Grouping algorithm. Our grouping algorithm favors simplicity, scanning the plan only once. This keeps planning fast, though the resulting grouping may be suboptimal. Avoiding the grouping of operators from different categories altogether, or identifying a better group frequency, are both interesting directions; however, we believe the solution space is limited by the relatively large time interval required for each group. We initially intended to base this time interval solely on the latency of frequency transitions, but the inaccuracy of the runtime estimator forced us to enlarge it further. With better runtime estimates, more fine-grained and precise grouping should be possible.

Capped frequencies vs. query-aware DVFS. Capping the GPU to a single frequency for the entire experiment performs surprisingly well. However, we want to emphasize that the frequencies used to cap the execution were specifically selected for having shown good performance in our initial experiments. In practice, even a statically capped frequency requires workload profiling to select an appropriate value, and the best static frequency already differs between our two examples. In contrast, our query-aware DVFS can handle both memory-constrained and memory-rich executions dynamically, and can adapt to changing queries or datasets, since it is based on the characteristics of individual operators and accounts for data size dynamically when estimating runtime, something static solutions cannot do.

Frequency scaling during result transfer. In our experiments, we did not enable throttling during the CPU-bound transfer of results from GPU to CPU because TPC-H queries have small result sets and transfer is thus very fast. However, throttling could easily be enabled. It would be beneficial for workloads with larger outputs, and correspondingly longer transfer times, offering a larger window in which to throttle and save energy.

Workload-agnostic DVFS. GEEPAFS achieves a comparable balance between energy savings and performance preservation, but falls short in energy savings overall. Its 15-second adaptation cycle and 200ms sampling periods are both far longer than the duration of many queries (let alone individual operators), making it difficult to adapt to a changing workload. Our database-specific solution, by contrast, predicts and plans ahead rather than reacting, and therefore does not suffer from this adaptation latency.

6 Conclusion and future work

OpPilot is the first query-aware GPU DVFS system designed specifically for GPU-accelerated databases. It considers the GPU usage and runtime of individual query operators, scales frequencies accordingly, and accounts for the overhead of frequency transitions. On the TPC-H workload, OpPilot achieves up to 40% energy savings compared to running without hardware tuning. We compare it against various baselines to characterize where the approach performs well and where it does not.

For future work, more accurate runtime estimation would allow more frequent frequency switching, and thus provide better energy savings with minimal performance penalty. Further, evaluating a more sophisticated grouping algorithm might shed light on whether complexity can beat simplicity. Moreover, while we account for some GPU idle time, additional idle periods remain unexplored (for instance, during CPU-based plan generation). These idle periods are, however, likely difficult to determine in a multi-query environment. Finally, it will be interesting to extend our approach to a multi-GPU environment.

Artifact

Artifacts and instructions to reproduce our experiments are available at https://github.com/OMichaud0/gpu-database-dvfs.

References

[1] Luiz André Barroso and Urs Hölzle. 2007. The Case for Energy-Proportional Computing. *Computer* 40, 12 (2007), 33–37. doi:10.1109/MC.2007.443

[2] BlazingDB. 2021. BlazingSQL is a lightweight, GPU accelerated, SQL engine for Python. https://github.com/BlazingDB/blazingsql

[3] Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. HetExchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 544–556. doi:10.14778/3303753.3303760

[4] Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. 2024. Reducing Energy Bloat in Large Model Training. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 144–159. doi:10.1145/3694715.3695970

[5] DuckDB Labs. 2026. DuckDB. https://duckdb.org/. Accessed: 2026-08-28.

[6] Henning Funke, Sebastian Breß, Stefan Noll, Volker Markl, and Jens Teubner. 2018. Pipelined Query Processing in Coprocessor Environments. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1603–1618. doi:10.1145/3183713.3183734

[7] Hao Gao and Nikolai Sakharnykh. 2021. Scaling Joins to a Thousand GPUs. In *International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures, ADMS@VLDB 2021, Copenhagen, Denmark, August 16, 2021*, Rajesh Bordawekar and Tirthankar Lahiri (Eds.). 55–64. http://www.adms-conf.org/2021-camera-ready/gao_adms21.pdf

[8] João Guerreiro, Aleksandar Ilic, Nuno Roma, and Pedro Tomás. 2019. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Comput.* 83, C (April 2019), 93–117. doi:10.1016/j.parco.2018.02.001

[9] Binglei Guo, Jiong Yu, Dexian Yang, Hongyong Leng, and Bin Liao. 2022. Energy-Efficient Database Systems: A Systematic Survey. *ACM Comput. Surv.* 55, 6, Article 111 (Dec. 2022), 53 pages. doi:10.1145/3538225

[10] Sebastian Götz, Thomas Ilsche, Jorge Cardoso, Josef Spillner, Thomas Kissinger, Uwe Aßmann, Wolfgang Lehner, Wolfgang E. Nagel, and Alexander Schill. 2014. Energy-Efficient Databases Using Sweet Spot Frequencies. In *2014 IEEE/ACM 7th International Conference on Utility and Cloud Computing*. 871–876. doi:10.1109/UCC.2014.142

[11] Hazar Harmouch and Felix Naumann. 2017. Cardinality estimation: An experimental survey. *Proceedings of the VLDB Endowment* 11, 4 (2017), 499–512.

[12] Yuto Hayamizu, Masaru Kitsuregawa, and Kazuo Goda. 2025. Proactive Energy Management in Database Systems. *SIGENERGY Energy Inform. Rev.* 4, 5 (April 2025), 154–159. doi:10.1145/3727200.3727223

[13] Bingsheng He, Mian Lu, Ke Yang, Rui Fang, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. 2009. Relational query coprocessing on graphics processors. *ACM Trans. Database Syst.* 34, 4, Article 21 (Dec. 2009), 39 pages. doi:10.1145/1620585.1620588

[14] Dong He, Supun C Nakandala, Dalitso Banda, Rathijit Sen, Karla Saur, Kwanghyun Park, Carlo Curino, Jesús Camacho-Rodríguez, Konstantinos Karanasos, and Matteo Interlandi. 2022. Query processing on tensor computation runtimes. *Proc. VLDB Endow.* 15, 11 (July 2022), 2811–2825. doi:10.14778/3551793.3551833

[15] HeavyAI. 2021. HeavyDB is an open source SQL-based, relational, columnar database engine for hybrid CPU/GPU systems. https://github.com/BlazingDB/blazingsql

[16] Shashikant Ilager, Rajeev Muralidhar, Kotagiri Rammohanrao, and Rajkumar Buyya. 2020. A Data-Driven Frequency Scaling Approach for Deadline-aware Energy Efficient Scheduling on Graphics Processing Units (GPUs). In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. 579–588. doi:10.1109/CCGrid49817.2020.00-35

[17] Tomas Karnagel, Dirk Habich, and Wolfgang Lehner. 2017. Adaptive work placement for query processing on heterogeneous computing resources. *Proc. VLDB Endow.* 10, 7 (March 2017), 733–744. doi:10.14778/3067421.3067423

[18] Thomas Kissinger, Dirk Habich, and Wolfgang Lehner. 2018. Adaptive Energy-Control for In-Memory Database Systems. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 351–364. doi:10.1145/3183713.3183756

[19] Mustafa Korkmaz, Alexey Karyakin, Martin Karsten, and Kenneth Salem. 2015. Towards Dynamic Green-Sizing for Database Servers. In *International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures - ADMS 2015, Kohala Coast, Hawaii, USA, August 31, 2015*, Rajesh Bordawekar, Tirthankar Lahiri, Bugra Gedik, and Christian A. Lang (Eds.). 25–36. http://www.adms-conf.org/2014/adms15_korkmaz.pdf

[20] Hai Lan, Zhifeng Bao, and Yuwei Peng. 2021. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. *arXiv preprint arXiv:2101.01507* (2021).

[21] Willis Lang and Jignesh M. Patel. 2009. Towards Eco-friendly Database Management Systems. In *Fourth Biennial Conference on Innovative Data Systems Research, CIDR 2009, Asilomar, CA, USA, January 4-7, 2009, Online Proceedings*. www.cidrdb.org. http://www-db.cs.wisc.edu/cidr/cidr2009/Paper_51.pdf

[22] Yinan Li, Bailu Ding, Ziyun Wei, Lukas M. Maas, Momin Al-Ghosien, Spyros Blanas, Nicolas Bruno, Carlo Curino, Matteo Interlandi, Craig Peeper, Kaushik Rajan, Surajit Chaudhuri, and Johannes Gehrke. 2025. Scaling GPU-Accelerated Databases Beyond GPU Memory Size. *Proc. VLDB Endow.* 18, 11 (July 2025), 4518–4531. doi:10.14778/3749646.3749710

[23] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2020. Pump Up the Volume: Processing Large Data on GPUs with Fast Interconnects. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1633–1649. doi:10.1145/3318464.3389705

[24] Xinxin Mei, Ling Sing Yung, Kaiyong Zhao, and Xiaowen Chu. 2013. A measurement study of GPU DVFS on energy conservation. In *Proceedings of the Workshop on Power-Aware Computing and Systems* (Farmington, Pennsylvania) *(HotPower '13)*. Association for Computing Machinery, New York, NY, USA, Article 10, 5 pages. doi:10.1145/2525526.2525852

[25] Sina Meraji, Berni Schiefer, Lan Pham, Lee Chu, Peter Kokosielis, Adam Storm, Wayne Young, Chang Ge, Geoffrey Ng, and Kajan Kanagaratnam. 2016. Towards a Hybrid Design for Fast Query Processing in DB2 with BLU Acceleration Using Graphical Processing Units: A Technology Demonstration. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1951–1960. doi:10.1145/2882903.2903735

[26] Hamish Nicholson, Aunn Raza, Periklis Chrysogelos, and Anastasia Ailamaki. 2023. HetCache: Synergising NVMe Storage and GPU acceleration for Memory-Efficient Analytics. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org. https://vldb.org/cidrdb/2023/hetcache-synergising-nvme-storage-and-gpu-acceleration-for-memory-efficient-analytics.html

[27] NVIDIA. 2025. NVIDIA Blackwell Ultra. https://resources.nvidia.com/en-us-blackwell-architecture/blackwell-ultra-datasheet

[28] NVIDIA. 2026. nvbandwidth: A tool for bandwidth measurements on NVIDIA GPUs. https://github.com/NVIDIA/nvbandwidth

[29] Iraklis Psaroudakis, Thomas Kissinger, Danica Porobic, Thomas Ilsche, Erietta Liarou, Pınar Tözün, Anastasia Ailamaki, and Wolfgang Lehner. 2014. Dynamic fine-grained scheduling for energy-efficient main-memory queries. In *Proceedings of the Tenth International Workshop on Data Management on New Hardware* (Snowbird, Utah, USA) *(DaMoN '14)*. Association for Computing Machinery, New York, NY, USA, Article 1, 7 pages. doi:10.1145/2619228.2619229

[30] Nazanin Rashedi and Guido Moerkotte. 2025. The accuracy of cardinality estimators: Unraveling the evaluation result conundrum. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3744–3756.

[31] Udhav Sethi. 2021. *Learning Energy-Aware Transaction Scheduling in Database Systems.* Master's thesis. University of Waterloo.

[32] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. 2020. A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1617–1632. doi:10.1145/3318464.3380595

[33] Mirsaeid Hosseini Shirvani, Amir Masoud Rahmani, and Amir Sahafi. 2020. A survey study on virtual machine migration and server consolidation techniques in DVFS-enabled cloud datacenter: taxonomy and challenges. *Journal of King Saud University-Computer and Information Sciences* 32, 3 (2020), 267–286.

[34] Sirius. 2026. Sirius source code. https://github.com/sirius-db/sirius

[35] Jeffrey Spaan, Kuan-Hsun Chen, and Ana-Lucia Varbanescu. 2026. Reducing Compute Waste in LLMs through Kernel-Level DVFS. arXiv:2601.08539 [cs.PF] https://arxiv.org/abs/2601.08539

[36] Substrait. 2026. Substrait: Cross-Language Serialization for Relational Algebra. https://substrait.io/. Accessed: 2026-08-28.

[37] Zhenheng Tang, Yuxin Wang, Qiang Wang, and Xiaowen Chu. 2019. The Impact of GPU DVFS on the Energy and Performance of Deep Learning: an Empirical Study. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems* (Phoenix, AZ, USA) *(e-Energy '19)*. Association for Computing Machinery, New York, NY, USA, 315–325. doi:10.1145/3307772.3328315

[38] Transaction Processing Performance Council. 2026. TPC-H Benchmark. https://www.tpc.org/tpch/. Accessed: 2026-08-28.

[39] Yi-Cheng Tu, Xiaorui Wang, Bo Zeng, and Zichen Xu. 2014. A system for energy-efficient data management. *SIGMOD Rec.* 43, 1 (May 2014), 21–26. doi:10.1145/2627692.2627696

[40] Daniel Velicka, Ondrej Vysocky, Osman Yasal, and Lubomir Riha. 2026. An in-depth study of GPU frequency-scaling latency and its optimization on modern architectures. *Future Generation on Computer Systems* 179 (2026), 108331. doi:10.1016/J.FUTURE.2025.108331

[41] Haicheng Wu, Gregory Diamos, Srihari Cadambi, and Sudhakar Yalamanchili. 2012. Kernel Weaver: Automatically Fusing Database Primitives for Efficient GPU Computation. In *Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture* (Vancouver, B.C., CANADA) *(MICRO-45)*. IEEE Computer Society, USA, 107–118. doi:10.1109/MICRO.2012.19

[42] Zichen Xu, Xiaorui Wang, and Yi-Cheng Tu. 2013. Power-Aware Throughput Control for Database Management Systems. In *10th International Conference on Autonomic Computing, ICAC'13, San Jose, CA, USA, June 26-28, 2013*, Jeffrey O. Kephart, Calton Pu, and Xiaoyun Zhu (Eds.). USENIX Association, 315–324. https://www.usenix.org/conference/icac13/technical-

sessions/presentation/xu_zichen

[43] Bobbi Yogatama, Weiwei Gong, and Xiangyao Yu. 2024. Scaling your Hybrid CPU-GPU DBMS to Multiple GPUs. *Proc. VLDB Endow.* 17, 13 (Sept. 2024), 4709–4722. doi:10.14778/3704965.3704977

[44] Bobbi Yogatama, Yifei Yang, Kevin Kristensen, Devesh Sarda, Abigale Kim, Adrian Cockcroft, Yu Teng, Joshua Patterson, Gregory Kimball, Wes McKinney, Weiwei Gong, and Xiangyao Yu. 2026. Rethinking Analytical Processing in the GPU Era. In *16th Conference on Innovative Data Systems Research, CIDR 2026, Chaminade, CA, USA, January 18-21, 2026*. www.cidrdb.org. https://vldb.org/cidrdb/2026/rethinking-analytical-processing-in-the-gpu-era.html

[45] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 119–139. https://www.usenix.org/conference/nsdi23/presentation/you

[46] Yuan Yuan, Rubao Lee, and Xiaodong Zhang. 2013. The Yin and Yang of processing data warehousing queries on GPU devices. *Proc. VLDB Endow.* 6, 10 (Aug. 2013), 817–828. doi:10.14778/2536206.2536210

[47] Shuhao Zhang, Jiong He, Bingsheng He, and Mian Lu. 2013. OmniDB: towards portable and efficient query processing on parallel CPU/GPU architectures. *Proc. VLDB Endow.* 6, 12 (Aug. 2013), 1374–1377. doi:10.14778/2536274.2536319

[48] Yijia Zhang, Qiang Wang, Zhe Lin, Pengxiang Xu, and Bingqiang Wang. 2024. Improving GPU Energy Efficiency through an Application-transparent Frequency Scaling Policy with Performance Assurance. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 769–785. doi:10.1145/3627703.3629584